

Earn-While-It-Loads — profiling questions during a slow load, paid in store credit

When a load is predicted to be slow, the app turns the wait into earning: it shows generic "know your customer" profiling questions and pays the user store credit for each answer, until whatever they're waiting on finishes. Works on web and on mobile (Android + iOS via the PWA/app shell). Prepared 2026-09-13.

1. What it does

If a page, screen, or data load is going to make the user wait, instead of a blank spinner they get a small card with a quick multiple-choice question. Each answer earns **1¢ of store credit** (a non-cashable Site Point), capped at **10¢/day**. The card keeps offering questions until the load completes, then disappears on its own. It has a close button — it's a "do something useful while you wait," never a trap.

2. The trigger is PREDICTIVE — it fires in advance

Per the directive, the questions appear **as soon as the app knows a load will exceed the load-speed budget** — not after a fixed wait, and *before* the load degrades enough that the resilient (on-device) fallback overrides the normal load. The prediction uses the same AI load-time signals the speed system already collects:

- **Server hint** (`predicted_slow`): the real-user p75 in-app navigation time is already over the perception budget (from `perf-optimizer.ts`).
- **Connection**: the device reports a slow connection (`slow-2g/2g/3g`) or data-saver.
- **Resilient state**: the platform is degraded/overloaded (the resilient-mode signal).

If any of those say "this will be slow," the card shows almost immediately (~250ms). If none do but the load turns out slow anyway, a fallback timer (`LOADING_SURVEY_TRIGGER_MS` , default 1200ms — deliberately set below the resilient override) still catches it. Either way it dismisses the instant the load finishes.

3. The 1,000 questions (non-sensitive by construction)

`backend/sdk/kyc-loading-questions.ts` holds **1,000 generic customer-profiling ("know your customer") questions** across ~50 topics: category interest and shopping frequency, price sensitivity, brand affinity, discovery habits, reward and communication preferences, device and timing, shopping style, and so on. They are **non-sensitive by design** — preferences and habits only. There is deliberately **no** identity, financial- account, health, or protected-class question, because a loading screen is the wrong place to collect sensitive personal data and none of it is needed for profiling. Each user is served questions they haven't answered recently, shuffled.

4. The reward is safe closed-loop credit, and can't be farmed

- The reward is **non-cashable Site Points (store credit)** — a closed-loop credit, **not a money action**, so it carries no money-transmitter exposure. It's booked as a bounded promotional **subsidy** (a cost).
- The **10¢/day cap is enforced server-side** (`LOADING_SURVEY_DAILY_CAP_POINTS`), so replaying the answer endpoint can't earn more than the cap — the cap *is* the anti-farming control. Answers beyond the cap are still recorded (the data is useful) but earn nothing.
- Anonymous visitors still see and can answer the questions (their answers are useful first-party data) but earn nothing until signed in.

5. The answers are first-party data that feed the platform

Every answer is stored as a `LoadingSurveyResponse` (first-party survey data: question, topic, answer). This is exactly the kind of profiling signal the personalization and data-driven optimization loops consume, so the feature does double duty — it makes waiting rewarding for the user *and* enriches the customer model the platform already builds from first-party data.

6. Compliance notes (for counsel / privacy)

- **Privacy disclosure.** Collecting customer-profiling answers is within the "survey participation and answers" first-party category the platform already discloses; the privacy policy should continue to cover this profiling use. It collects no new *category* of sensitive data.
- **Reward.** Non-cashable, closed-loop, capped — consistent with the rest of the closed-loop points model.
- **Not sensitive KYC.** "Know your customer" here means marketing/customer profiling, **not** identity verification — no identity or financial data is requested.

7. Settings (Automation; on from day one)

`LOADING_SURVEY_ENABLED` (default on), `LOADING_SURVEY_PREDICT_ENABLED` (predictive trigger, default on),
`LOADING_SURVEY_REWARD_POINTS` (1 = 1¢), `LOADING_SURVEY_DAILY_CAP_POINTS` (10 = 10¢/day),
`LOADING_SURVEY_BATCH_SIZE` (questions pre-fetched), `LOADING_SURVEY_TRIGGER_MS` (fallback wait).

8. Where it lives (for the reviewer / developer)

- Questions: `backend/sdk/kyc-loading-questions.ts` (1,000).
- SDK: `backend/sdk/loading-survey.ts` (pick, predict, record + capped reward).
- Functions: `loadingSurveyNext` (batch + trigger config, auth-optional), `loadingSurveyAnswer` (record + award).
- Data: `LoadingSurveyResponse` entity (first-party answers).
- Client: `src/lib/loading-survey.js` (predictive controller), `src/components/LoadingSurveyOverlay.jsx` (the card, driven by React Query's active-fetch count), wired in `src/App.jsx` + `src/main.jsx`.